

Machine Learning para Finanzas

Clase 8

César Núñez Cuevas

`cnunezc@fen.uchile.cl`



UNIVERSIDAD

Finis Terrae

1. El Paradigma de Aprendizaje por Refuerzo

El Aprendizaje por Refuerzo (RL) difiere del aprendizaje supervisado y no supervisado.

- No aprende de un conjunto de datos estático con etiquetas correctas.
- No busca patrones ocultos en datos sin etiquetar.
- **Esencia:** Aprender a través de la interacción directa, el ensayo y el error con el fin de maximizar una recompensa acumulada en el tiempo.

2. El Ciclo de Interacción del RL

El proceso se describe como un bucle continuo y dinámico en pasos de tiempo discretos t :

- 1 El agente observa el Estado actual s_t del entorno.
- 2 El agente ejecuta una Acción a_t basándose en su estrategia.
- 3 El entorno cambia a un nuevo estado s_{t+1} debido a dicha acción.
- 4 El entorno devuelve una Recompensa r_{t+1} como retroalimentación.

3. Procesos de Decisión de Markov (MDP)

Formalismo matemático que sustenta el RL. Un MDP se define por la tupla $\langle S, A, P, R, \gamma \rangle$:

- **Propiedad de Markov:** El futuro depende únicamente del estado presente, no del historial pasado.

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$$

- Esto simplifica radicalmente el modelado matemático de entornos complejos.

4. La Recompensa Acumulada y el Retorno

El agente no solo busca el beneficio inmediato, sino maximizar el Retorno (G_t):

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Donde $\gamma \in [0, 1]$ es el factor de descuento:

- Si $\gamma \rightarrow 0$, el agente es "miope" (solo le importa el corto plazo).
- Si $\gamma \rightarrow 1$, el agente es "visionario" (valora enormemente recompensas futuras).

5. ¿Cómo entrena un modelo de RL?

El entrenamiento es un proceso de optimización iterativo:

- **Recolección de Experiencia:** El agente recopila trayectorias de datos en forma de tuplas $(s_t, a_t, r_{t+1}, s_{t+1})$.
- **Actualización de Estructuras:** Utiliza esos datos para corregir sus estimaciones de valor o su mapa de decisiones.
- **Convergencia:** El bucle se repite millones de veces hasta que las decisiones del agente se estabilizan en la estrategia óptima.

6. El Agente (Agent)

El Agente es la entidad inteligente, el motor de toma de decisiones.

- Contiene el código, la red neuronal o la tabla matemática que procesa la información.
- Su único canal de comunicación con el exterior es observar estados y emitir acciones.
- **Ejemplo Financiero:** Un algoritmo automatizado de arbitraje o un gestor de portafolios.

7. El Entorno (Environment)

El Entorno es el sistema, mundo o simulación externa al agente.

- Responde a las acciones del agente siguiendo leyes físicas, económicas o reglas lógicas prefijadas.
- Es dinámico y frecuentemente estocástico (con un grado de aleatoriedad).
- **Ejemplo Financiero:** El libro de órdenes de la bolsa de valores o el sistema de simulación macroeconómica.

8. El Estado (State - s)

El Estado es la fotografía completa o parcial del entorno en un momento dado.

- Contiene toda la información relevante para que el agente tome una decisión informada.
- Distinguimos entre estado completamente observable (ajedrez) y parcialmente observable (póker, mercados financieros).
- **Ejemplo Financiero:** Vector con precios históricos, medias móviles, volatilidad y balance de caja actual.

9. La Acción (Action - a)

La Acción representa el conjunto de decisiones que el agente tiene permitido ejecutar.

- Puede pertenecer a un espacio de acciones discreto (ej. comprar, vender, mantener) o continuo (ej. el porcentaje exacto de capital a asignar a un activo).
- El mapa de acciones disponibles puede cambiar según el estado actual.

10. La Recompensa (Reward - r)

La Recompensa es la señal numérica escalar que evalúa el éxito inmediato de la acción.

- El diseño de la función de recompensa (Reward Shaping) es el desafío más complejo en RL.
- Una recompensa mal estructurada puede provocar comportamientos imprevistos o tramposos por parte del agente.
- **Ejemplo Financiero:** El rendimiento ajustado por riesgo (Ratio de Sharpe) al final de una jornada.

11. El Dilema de Exploración vs. Explotación

Es el equilibrio y problema central que debe gestionar todo algoritmo de RL:

- **Exploración:** Probar acciones desconocidas o subóptimas para descubrir mejores estrategias a largo plazo.
- **Explotación:** Utilizar el conocimiento actual acumulado para elegir la acción que genera la mayor recompensa conocida.
- Explotar permanentemente lleva a estancarse en óptimos locales; explorar indefinidamente malgasta recursos sin acumular ganancias.

12. Estrategia ϵ -greedy (Épsilon-Codiciosa)

La solución más común y robusta para balancear la exploración:

$$a_t = \begin{cases} \text{Acción aleatoria} & \text{con probabilidad } \epsilon \\ \arg \max_a Q(s_t, a) & \text{con probabilidad } 1 - \epsilon \end{cases}$$

- **Decaimiento de ϵ :** Se inicia con un ϵ alto (ej. 1.0) para explorar agresivamente al inicio, y se reduce gradualmente (ej. hasta 0.01) a medida que el agente se vuelve experto.

13. La Política (Policy - π)

La Política representa el mapa de comportamiento del agente. Es su estrategia formal.

- **Política Determinista:** Mapea un estado directamente a una acción específica: $a = \pi(s)$.
- **Política Estocástica:** Mapea una distribución de probabilidad sobre el conjunto de acciones dado un estado: $\pi(a | s) = P(a_t = a | s_t = s)$.

14. Funciones de Valor

Miden qué tan rentable a largo plazo es encontrarse en un estado determinado.

- **Función de Valor de Estado $V^\pi(s)$:** El retorno esperado partiendo del estado s siguiendo la política π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

- **Función de Valor de Acción $Q^\pi(s, a)$:** El retorno esperado tras realizar la acción a en el estado s :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

15. Ecuaciones de Esperanza de Bellman

Las funciones de valor pueden descomponerse de forma recursiva:

$$Q^{\pi}(s, a) = \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q^{\pi}(s', a') \right]$$

- Relaciona el valor del estado-acción actual con el valor esperado de los estados-acciones subsiguientes.
- Es el pilar fundamental que permite diseñar algoritmos de programación dinámica y aprendizaje temporal.

16. Métodos Tabulares

Se utilizan cuando los espacios de estados y acciones son pequeños y discretos.

- El conocimiento se almacena en una Tabla Q (Q-Table), donde las filas representan los estados y las columnas las acciones.
- Cada celda contiene el valor numérico estimado $Q(s, a)$.
- El objetivo es actualizar iterativamente estas celdas hasta aproximarse a los valores reales óptimos.

17. Q-Learning (Algoritmo Off-Policy)

Encuentra la política óptima independientemente de las acciones tomadas por el agente durante la exploración.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **Off-Policy:** Utiliza el operador $\max_{a'}$ asumiendo que en el siguiente estado tomará la mejor acción posible, ignorando la política real de exploración ϵ -greedy.
- α representa la tasa de aprendizaje.

18. SARSA (Algoritmo On-Policy)

El nombre proviene de la tupla involucrada en su actualización:
State-Action-Reward-State-Action.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

- **On-Policy:** Utiliza la acción real a' seleccionada por la política actual (incluyendo el factor de riesgo o exploración).
- **Diferencia con Q-Learning:** SARSA es más seguro en entornos penalizados, ya que modela el peligro intrínseco de la exploración activa.

19. Deep Q-Networks (DQN)

¿Qué ocurre si el entorno tiene infinitos estados continuos? La tabla Q se vuelve inviable.

- DQN sustituye la tabla Q por una Red Neuronal Artificial parametrizada por pesos θ : $Q(s, a; \theta)$.
- La red toma el estado como entrada y devuelve el valor Q estimado para cada acción disponible.
- Incorpora trucos clave como el Experience Replay y redes objetivo fijas para estabilizar la convergencia.

20. Proximal Policy Optimization (PPO)

Un algoritmo avanzado dentro de la familia de Métodos de Gradiente de Política (Policy Gradient).

- En lugar de modelar los valores Q , optimiza directamente los parámetros de la política $\pi(a | s)$.
- PPO introduce una función objetivo recortada (clipping) para evitar cambios drásticos y destructivos en la estrategia durante una actualización.
- Es el estándar moderno en robótica y en el alineamiento de grandes modelos de lenguaje (RLHF).

21. Model-Free vs. Model-Based RL

Clasificación según la estructura del conocimiento del agente:

- **Model-Free (Libre de Modelo):** El agente no intenta comprender las transiciones del entorno. Aprende puramente de la experiencia y la recompensa. (Ej. Q-Learning, DQN).
- **Model-Based (Basado en Modelo):** El agente construye o aprende un "mapa mental" de la dinámica del entorno $P(s' | s, a)$. Le permite simular y planificar con anticipación antes de actuar (Ej. MuZero).

22. Ventajas del Enfoque Model-Based

- Eficiencia de Muestreo: Requiere interactuar muchas menos veces con el entorno real, ya que puede "soñar.^o planificar en su modelo mental simulado.
- En finanzas, esto reduce el riesgo de perder dinero real durante fases iniciales de entrenamiento algorítmico.

23. Multi-Agent Reinforcement Learning (MARL)

Estudia entornos donde coexisten e interactúan múltiples agentes simultáneamente.

- **Entornos Cooperativos:** Los agentes comparten un objetivo común y maximizan una recompensa conjunta.
- **Entornos Competitivos:** Juego de suma cero, el beneficio de un agente implica la pérdida de otro.
- **Desafío Técnico:** El entorno se vuelve no estacionario desde la perspectiva de un solo agente, rompiendo la propiedad de Markov.

24. MARL en Sistemas Financieros

El mercado real no está compuesto por un solo algoritmo, sino por miles de ellos operando a la vez.

- Modelar el mercado como un sistema multi-agente permite analizar dinámicas agregadas complejas.
- Permite estudiar la aparición de caídas repentinas (flash crashes) creadas por la retroalimentación competitiva de algoritmos de alta frecuencia.

25. Desafíos Generales del Aprendizaje por Refuerzo

- **Recompensas Dispersas (Sparse Rewards):** Si el agente pasa horas sin recibir una señal clara de éxito, no sabrá qué optimizar.
- **Inestabilidad de las Redes:** La combinación de aproximadores de funciones no lineales con RL suele ser propensa a divergencias numéricas.

26. El Agente de Trading Automatizado

La aplicación icónica en finanzas cuantitativas.

- **Estado:** Datos del libro de órdenes, indicadores técnicos, posición de inventario actual.
- **Acciones:** Enviar orden de compra límite, orden de mercado, cancelar orden.
- **Recompensa:** Ganancia neta realizada corregida por una penalización por riesgo o volatilidad del portafolio.

27. Optimización de Portafolios Dinámicos

Asignación continua de pesos de capital en una cesta de activos.

- A diferencia de la teoría clásica de Markowitz (que es estática), un agente de RL puede rebalancear la cartera diariamente adaptándose a las condiciones cambiantes del mercado.
- Utiliza algoritmos de acción continua (como DDPG o PPO) para definir los porcentajes exactos de inversión.

28. Ejecución Óptima de Órdenes Grandes

Cómo vender un paquete masivo de acciones sin hundir el precio de mercado (Market Impact).

- El agente aprende a fragmentar la orden gigante en micro-órdenes estratégicas a lo largo del día.
- **Recompensa:** Minimizar el *slippage* (la diferencia entre el precio esperado y el precio finalmente ejecutado).

29. Gestión del Riesgo de Crédito y Cobranzas

- Un agente de RL puede entrenarse para decidir de manera personalizada el canal de contacto, la oferta de refinanciación y el momento idóneo para interactuar con clientes en mora.
- **Meta:** Maximizar el capital recuperado minimizando el costo operativo de las llamadas o acciones legales.

30. Síntesis y Mejores Prácticas

- Simuladores de Alta Fidelidad: Nunca expongas un algoritmo de RL crudo al mercado real; requiere fases masivas de entrenamiento en entornos simulados validados estadísticamente.
- Simplicidad Primero: Si una regla heurística o un modelo lineal supervisado resuelve el problema eficientemente, evite la complejidad computacional implícita en el Reinforcement Learning.